

Job Intelligence Platform v4: Claude Code Handoff Roadmap

User: Joey Clark **Repo:** github.com/jojo-d-explorer/ai-automation-portfolio (local: `~/GitHub/ai-automation-portfolio`) **Written:** 2026-07-19 **Execution environment going forward:** Claude Code (any current model: Opus, Sonnet). This document assumes no Fable-specific capabilities. Everything here is executable by any competent coding agent with shell access.

1. Context for the executing agent

You are picking up a mature, working system, not a greenfield project. Read the repo README and `Core/friend_search_operations.md` first. The system has run weekly since February 2026 for five users. Joey's own search is the R&D track.

Joey's current search parameters (authoritative as of July 2026):

- Role families: Chief of Staff; Country Manager / GM (Mexico, Argentina, LATAM); Head/Director/VP Partnerships; Senior BD; Market Entry / International Expansion; Director+ Operations / BizOps; Strategic Initiatives; Corporate Development; Operating Partner / VC Platform / Portfolio Ops; Ecosystem Lead; Government Affairs / Institutional Partnerships.
- Geography, in priority order:
 - Tier 1: Mexico City / CDMX, Buenos Aires, Argentina, Mexico
 - Tier 2: Remote-LATAM, Remote-Americas, Remote-Global (no US-only qualifier). A role counts only if it is genuinely open remote: no US residency or work-location requirement, no state restriction, no hybrid or recurring-onsite pattern. Roles labeled "Remote (US)" are excluded by the pipeline. Rationale (decided 2026-07-19): Joey will be based in Buenos Aires or Mexico City and will not misrepresent his location; US-restricted remote roles end in frustration. Exception path: if Joey has a warm connection at a company that would waive the restriction, he adds that company to the corpus manually; the pipeline does not guess at exceptions.
 - Tier 3: Remote-Global with explicit LATAM expansion mandate.
 - Hard exclude: in-person-locked non-LATAM cities, US-residency-restricted roles.

- Sectors (priority order): fintech/payments, EOR/cross-border ops, defense/dual-use, energy/industrial tech, VC/PE platform with LATAM mandate.
- Seniority: Director/VP/Head/CoS/Country Manager. Two-dimension-stretch rule applies (see scoring, section 5).
- Spanish: beginner, actively learning. Spanish-required roles get flagged, not excluded.
- Resume: `resume/Joey_Clark_Resume_4-25_FINAL.pdf` (use LATAM-adapted variant if present in repo).

Known failure modes, diagnosed from run history. Do not reintroduce these:

1. **Stale-index discovery.** Google site: searches of ATS boards return listings that are dead by fetch time. One run: 121 of 373 working-set rows (32%) were already closed. Root fix was API-first (query Greenhouse/Lever/Ashby board APIs directly). Search-sourced rows remain the contamination vector.
2. **Snippet scoring.** Scoring against search-result snippets instead of full JDs produced a 42/42 templated Master with unreliable scores. Rule: no score is assigned without fetched JD text. Rows that fail JD fetch get a fetch-failure status, never a snippet-based score.
3. **Runtime coupling.** The v3.x pipeline was a prompt that told an agent which tools to call (web_fetch vs Claude-in-Chrome navigate). When the runtime changed, the pipeline silently degraded. The v4 fix is architectural: deterministic Python does all fetching and verification; the model never touches a network primitive for pipeline-critical steps.
4. **Late URL verification.** `check_urls.py` ran after scoring, so Stage 2 budget burned on dead URLs (one run: 59% of Master rows removed by the final health check). v4 moves health checks before JD fetch.
5. **LATAM recall gap.** Country Manager / market entry roles in LATAM post disproportionately to LinkedIn, Getonbrd, and direct career pages rather than US-centric ATS boards. The corpus and source list must reflect this.

2. The core architectural change

v3.x: prompt-as-program. A markdown prompt drives an agent through search, fetch, scoring, and file output in one session. Fragile, model-dependent, expensive.

v4: code-as-pipeline, model-as-judge.

- Python owns everything deterministic: corpus querying, discovery, URL health, JD

fetching (requests + Playwright fallback for JS-rendered pages), dedup, NEW/REPEAT, CSV/XLSX build, metrics.

- The model owns only judgment: scoring fetched JD text against the rubric and resume, writing rationales, extracting soft fields (Spanish requirement, remote-mode nuance, builder-vs-manager signals), and the weekly synthesis note.
- The judgment step is invoked either interactively in Claude Code or headlessly (`claude -p` / Anthropic API) with a fixed scoring prompt. Any model can run it because the input is plain JD text plus a rubric.

This makes the pipeline model-agnostic, cheaper (the model reads ~40 JDs a week instead of orchestrating hundreds of tool calls), and debuggable (every intermediate is a file on disk).

One command target: `python3 JC3/run.py --user joey --variant latam` executes discovery through packaged XLSX, pausing once for the model scoring step (or calling it headlessly if configured).

3. Phase 0: Handoff hygiene (first session, ~1-2 hours)

Acceptance criteria at the end of each phase. Do these in order.

1. **Write** `CLAUDE.md` **at repo root** containing: the one-paragraph system description, Joey's search parameters (section 1 above), the five failure modes as standing rules, file layout conventions, and the run commands. This is the persistent context for every future Claude Code session. Keep it under 200 lines.
2. **Create** `config/joey_profile.yaml` consolidating everything currently scattered across ONE_CLICK markdown files: role families with title-gate keywords, location tiers, sector weights, scoring rubric weights, disqualifiers, score cutoff (60 for LATAM variant, flag 50-59), Spanish flag rule, comp floor. The pipeline reads config from here; prompts no longer carry config.
3. **Archive the Fable-era prompts.** Move ONE_CLICK v3.x/LATAM markdown files to `searches/archive/`. They remain reference, not runbooks.
4. **Verify the environment:** `python3 ats_api_checker.py` runs clean against Joey's `companies.json`; `pip install playwright` and `playwright install chromium` succeed; git status clean.

Acceptance: `CLAUDE.md` exists, `profile.yaml` validates (write a tiny loader that fails loudly on missing keys), API checker runs, one commit.

4. Phase 1: Deterministic core (week 1)

Build `JC3/run.py` orchestrating four scripts (refactor existing code where it exists, do not rewrite what works):

1. `discover.py`

- Primary: full-corpus ATS API sweep (Greenhouse, Lever, Ashby) filtered by role-family keywords and location tiers. Weight companies tagged `latam_relevance: direct` (add this field to `companies.json`; default `none`, backfill from the sector-focus list in the archived LATAM ONE_CLICK).
- Secondary: SmartRecruiters public postings API (`api.smartrecruiters.com/v1/companies/{id}/postings`) for corpus companies on that platform. This converts a chunk of the 176 currently non-queryable companies into API coverage.
- Tertiary (small, capped): targeted web discovery only for finding NEW companies to feed the corpus, not for sourcing rows directly. New companies pass the existing disqualifier filter via `corpus_append.py`.
- Output: `candidates_{date}.csv` with source provenance per row (`api / smartrecruiters / discovery`).

2. `health.py` (refactor of `check_urls.py`): runs BEFORE JD fetch, on non-API rows only. API rows are live by definition. Adds a staleness gate: posted date > 45 days and non-API-verified gets dropped to a diagnostics file.

3. `fetch_jds.py` : requests first; on empty/short body or known JS-rendered domains (Workday, Oracle Cloud, Notion-hosted, Revolut-style pages), fall back to Playwright headless Chromium with a rendered-DOM text extraction. Per-row statuses: success / closed_listing / http_error / empty / js_shell. Store JD text as `jd_cache/{hash}.txt`. Target: 80%+ success rate; below that, halt and report rather than degrade.

4. `package.py` : dedup (Company + Job_Title, keep highest score, merge Found_On), NEW/REPEAT against Master_LATAM_Joey.csv, sort, write CSV + 3-tab branded XLSX using the existing `openpyxl` code, update Master.

Acceptance: one end-to-end run producing candidates with cached JD text and zero dead URLs in the survivor set, before any scoring exists.

5. Phase 2: Judgment layer (week 1-2)

1. `prompts/score_jd_v4.md` : a fixed, model-agnostic scoring prompt. Inputs: JD text, a 1-

page resume digest (generate once from the resume PDF, store as `config/joey_resume_digest.md`), the rubric from `profile.yaml`. Output: strict JSON per job: `{score, subscores{skills,geo,seniority,sector}, rationale, spanish_requirement, location_type, salary_usd, red_flags, stretch_dimensions[]}`.

- Encode the hiring filter: identify which of Function / Domain / Level / Geography / Compensation are stretches. Two or more stretches caps the score at 55 regardless of subscores.
 - Encode the four skip archetypes (partnerships-with-quota disguise, founding-sales-as-CoS, PE roll-up dev-track CoS, level-inverted CoS) as named `red_flags`.
 - Rubric weights per the LATAM variant: skills 35, geography 30, seniority 20, sector 15. Keep the geographic sub-bands from the June spec. Per the 2026-07-19 decision, US-restricted remote roles are excluded upstream at discovery/JD-verification, so the geo band never scores them; the JD-level remote-mode check (which catches “remote but US-only” language that snippets hide) is the enforcement point.
2. **score.py** : iterates `jd_cache`, calls the model (headless `claude -p --output-format json` or Anthropic API; make the backend a config option), validates JSON, writes scores into the working CSV. Batch of ~5 JDs per call is a good cost/quality balance.
 3. **Anti-hallucination invariant**: `score.py` only ever writes scores for rows with `jd_fetch = success`. Everything else carries its fetch status forward visibly.

Acceptance: a full weekly run where every scored row has a cached JD, rationales reference actual JD language, and two-stretch roles are visibly capped.

6. Phase 3: LATAM recall expansion (week 2-3)

Recall is the current binding constraint for the LATAM track. In priority order:

1. **Getonbrd integration**. Getonbrd (getonbrd.com) is the dominant LATAM tech job board and exposes a public API. Add `getonbrd.py` as a discovery source filtered to the role families and Mexico/Argentina/remote. Verify current API terms and endpoints at build time before coding against memory.
2. **Fund-portfolio corpus miner**. Using `latam_fund_corpus_v1.md` and the US-LATAM addendum in project knowledge: for each fund (Kaszek, monashees, ALLVP, Valor, Atlantico, NXTP, Nazca, QED, SoftBank LatAm, Endeavor Catalyst), pull the portfolio page, extract company names, resolve ATS slugs with `verify_ats.py` auto mode, append

with `latam_relevance: direct`. Expect this to add 100+ companies. This is the compounding lever; the corpus is the recall engine.

3. **EOR country teams.** Deel, Multiplier, Remote.com, G-P, Rippling: their own Mexico/Argentina country and expansion teams are direct fits. Ensure all are in-corpus and tagged.
4. **LinkedIn stays manual by design.** Keep `linkedin_links.py` generating tiered saved-search URLs (the March decision to remove LinkedIn from automation was correct; its index is unscreapeable reliably). Add the five-country LATAM set to the generated links. Joey reviews these by hand weekly; the pipeline does not pretend to cover them.
5. **Newsletter intel continues** (Scenius LATAM, FinSMEs) via the existing Gmail mining pattern, feeding `corpus_append.py`. Sender-based Gmail queries, not keyword queries.

Acceptance: corpus grows past 550 with `latam_relevance` tags; a weekly run surfaces Tier 1 (CDMX/BA-based) rows, which the June-July runs rarely did.

7. Phase 4: Operations (ongoing)

- **Cadence:** one weekly run, LATAM variant primary, RemoteGlobal as a second `--variant` invocation of the same pipeline (same code, different profile section). Retire the old Weekly Search variant.
- **Metrics file per run** (`results/joey/LATAM/Week_of_X/metrics.json`): rows by source, JD fetch success rate, dead-at-delivery count (target: 0), NEW rate, Tier 1 count, Spanish-required count, runtime. Trend these; they are the honest scoreboard and portfolio evidence.
- **Review loop:** every 4-6 weeks, Joey reviews scored-vs-applied divergence and adjusts rubric weights in `profile.yaml`. No automated rubric drift.
- **Execution home (decided):** Joey's Mac, weekly. Default trigger is manual from a Claude Code session; adding a `launchd` job for the deterministic stages (discover, health, fetch) is optional Phase 4 polish, with the judgment step run when Joey opens the session.
- **Friends are frozen (decided):** the v5.x production track is untouched indefinitely. Joey's profile is the only active user of v4. Do not modify anything under `searches/For_Others/` or `results/For_Others/`.
- **End state:** the system runs at weekly cadence until Joey accepts an offer. On acceptance: final metrics rollup, README write-up of the v4 arc as the portfolio capstone, archive.

- **Portfolio note:** commit messages and the README week-by-week log continue. The v3-to-v4 refactor (prompt-as-program to code-plus-judge) is itself a strong portfolio narrative; write it up in the README when stable.
-

8. Decisions resolved and reserved

Resolved 2026-07-19:

- Remote-US: excluded unless genuinely open remote (section 1). Warm-connection exceptions handled manually via corpus additions.
- Execution: Joey's Mac, weekly, until an offer is accepted.
- Friends: frozen on v5.x; Joey-only build.
- Scoring backend: default to headless `claude -p` inside Claude Code sessions. API scoring (Sonnet) is an approved fallback for scheduled/unattended runs; at ~40 JDs a week the API cost is a few dollars a month, well under any subscription-level spend, so cost is not a blocker if it improves reliability.

Still reserved for Joey (do not decide autonomously):

1. Compensation floor for scoring, and specifically whether CDMX/BA-based roles paying LATAM-local bands are acceptable or whether the search targets US-level comp regardless of base city. This affects the comp subscore and the two-stretch calculus.
2. Any change to the 60-point cutoff or rubric weights.
3. Whether the RemoteGlobal secondary variant runs weekly alongside LATAM or biweekly.

9. First-session checklist for Claude Code

1. Read CLAUDE.md (or create it: Phase 0.1)
2. Read this file top to bottom
3. `git pull`; confirm clean tree
4. Run Phase 0 items 2-4
5. Propose the Phase 1 file plan as a short diff-level outline, get Joey's go, bu

End of roadmap.